

SIMULATION TOOLBOX

Gearing up for robotics

Adam Dąbrowski

ROSCon 2026 workshop

THIS SESSION

Toolbox

Learn which simulators fit your problem and why.

Standards

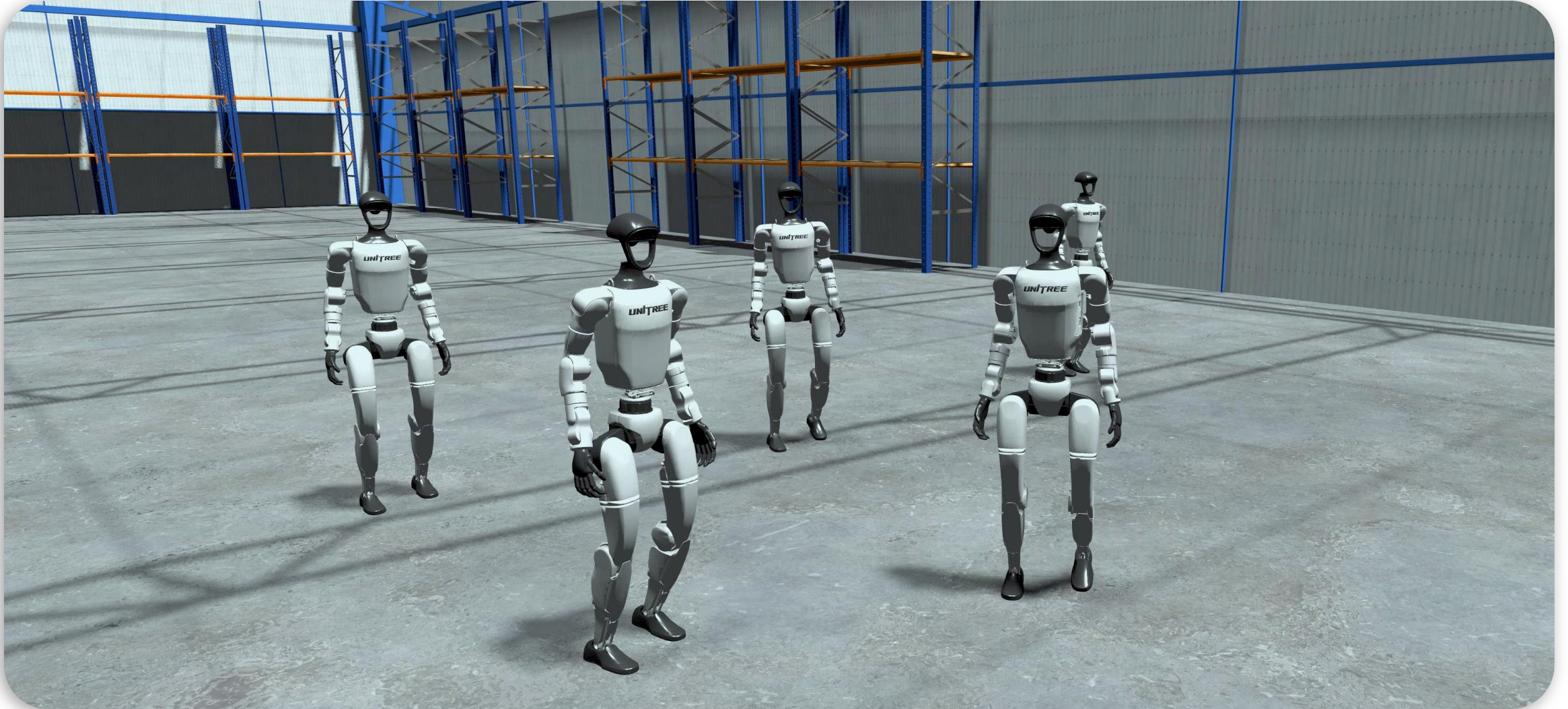
Reuse simulation workloads and assets.

Practice

Fine-tune a humanoid walking policy and take the resources home.



SHOW OF HANDS



NOTEBOOK RUN

Start the prepared training job.

Launch it and we come back to it 25 minutes later.

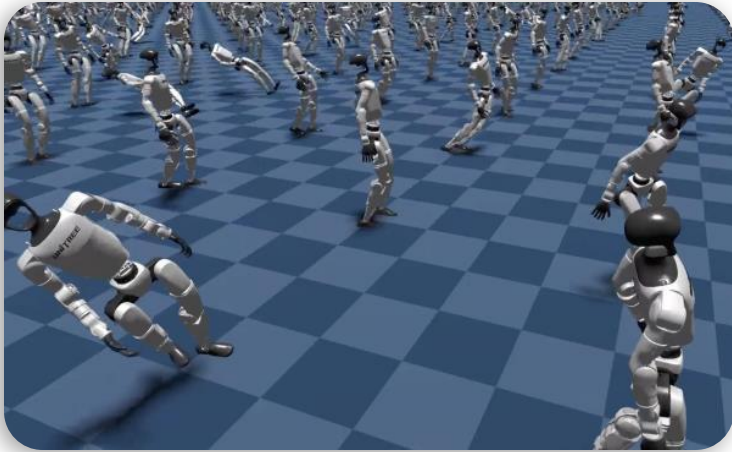
ROSCon Humanoid Locomotion Workshop

Welcome to the workshop hosted by **AMD** and **Robotec.ai**, running on an **AMD Strix Halo mini-PC**. We will use GPU-accelerated simulation and reinforcement learning to adapt a Unitree G1 humanoid policy from flat-ground walking to stair climbing.

The two workshop notebooks

1. [Humanoid locomotion — training](#) configures the stair terrain, rewards, curriculum, and PPO fine-tuning run. Start it first in its own Jupyter kernel and let it train in the background. It writes the result to `logs/roscon_stairs/model_stairs.pt`.
2. [Humanoid locomotion — evaluation](#) visualizes the generated terrain, measures the flat-ground baseline, plays its failure case, and then evaluates and plays the stair-trained policy.

INCREASING ROLE OF SIMULATION



Train

policies & models



Prototype

fail fast



Test ops

behavior at scale, with metrics

← *the shift in emphasis*

SIMULATION PURPOSES

Design and development

Prototype your software, quickly iterate; AI code engineering with sim in the loop.

Stack evaluation

Tasks, scenarios and benchmarks across the full stack.

Optimization

Improve by testing out possibilities, with event-driven or physics-based models.

Specialized models

Component simulation, degradation and predictive maintenance.

Robot learning

Scale training for walking, grasping and other skills.

Synthetic data

Generate sensor data for perception models.

Look-ahead simulation

Predict outcomes while the real robot is running.

Customer demonstrations

Show the customer's site looking happy with your robots operating in it.

SOFTWARE-IN-THE-LOOP AND ROBOT LEARNING

	Software-in-the-loop	Robot learning
Goal	Replace hardware for development and full use-case testing	Teach a skill, usually with an isolated task
Performance	Scene complexity, sensor fidelity and volume, robot count	Time to train, GPU batching and parallel environments
APIs	ROS, sockets and drivers; distributed communication	In-memory tensors; no middleware in the rollout loop
Time	Simulation drives the clock for the application stack	The learning algorithm steps the simulation
Scaling	Headless test jobs across cloud workers; real-time factor	Vectorized environment instances on the GPU

ARCHITECTURAL TRADE-OFFS

Software-in-the-loop

Long-running operations with interacting subsystems, distributed software and authored environments.

An RL wrapper does not provide GPU batching, fast resets or scalable training.

Robot learning

Episodes, repeated resets and many parallel copies of a focused task.

Scaling a learning task to an operating factory also needs scene authoring, behaviors and APIs.

Most simulators favor one of these workloads.

DIFFERENTIABLE PHYSICS

The simulator returns not just the outcome, but the derivative of the outcome with respect to every input.

Applications

System identification and real-to-sim, trajectory optimization over deformables and fluids, design co-optimization: fit or optimize thousands of continuous parameters by gradient descent instead of trial and error.

Learning relationship

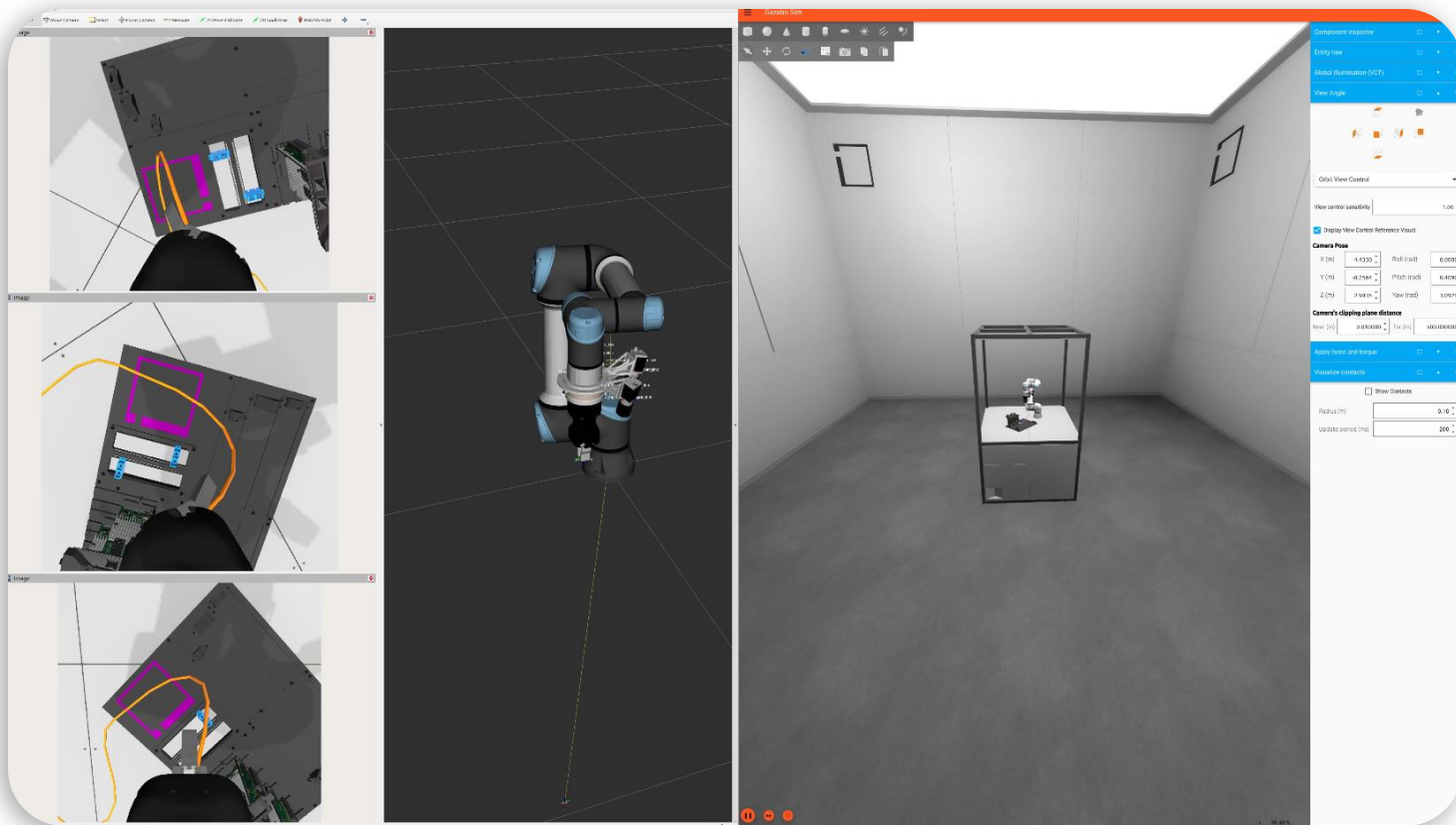
Rollout RL treats the simulator as a black box and needs no gradients; our humanoid fine-tuning is that case. Backpropagating reward through the physics is possible, sample-efficient, and fragile.

Practical limits

Hard contacts break gradients, long horizons gradients stop meaning anything: small perturbations grow exponentially.
Genesis World and Newton expose gradients, Gazebo / PhysX don't.

GAZEBO

Jetty LTS



Great support for ROS packages

Pairs with ROS 2 Jazzy and later.
Most ROS packages ship Gazebo examples; `ros_gz` bridges topics and services

Composable simulation

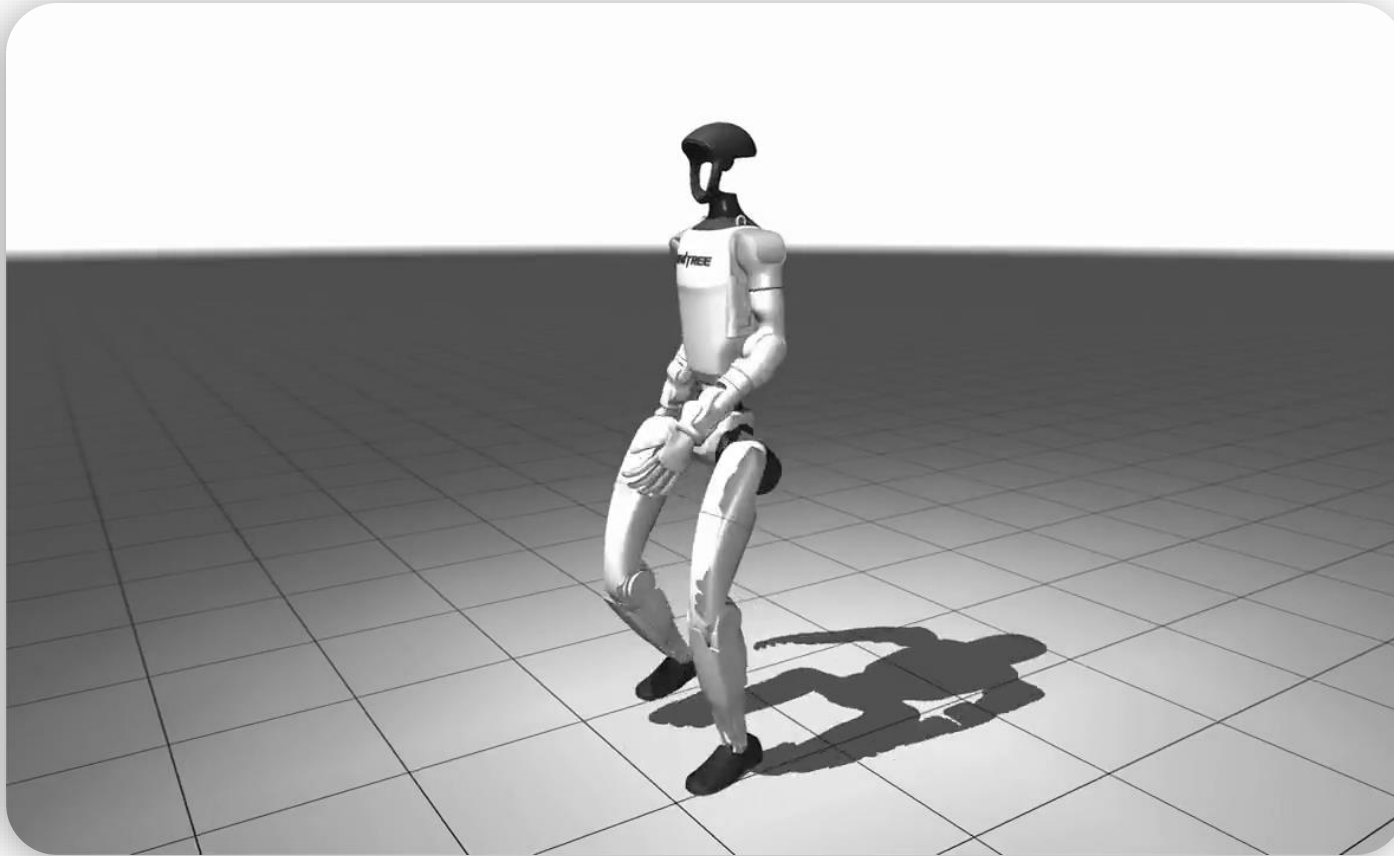
Swap physics, rendering and sensors as plugins.

Community standard

SDFFormat, Fuel asset library, Python bindings; Zenoh transport in Jetty for ROS 2 interoperability.

MuJoCo

3.13.0, September 2026



Control mechanics

Contacts, actuators, tendons and constraints suit locomotion and manipulation.

A ready learning stack

Playground provides tasks and training examples; Menagerie provides robot models.

Execution choices

CPU MuJoCo, MJX/JAX and MuJoCo Warp differ in supported features and hardware.

ISAAC SIM

6.1, September 2026



Sensors and physics

RTX cameras, lidar and radar; PhysX or Newton (MuJoCo Warp) behind one API. Isaac Lab trains on the same stack



ROS 2 in the loop

Bridge for Humble and Jazzy. 6.1 adds an in-process ros2_control manager.



Hardware

Needs an RTX GPU with RT cores (min. RTX 4080, 16 GB); no CPU or other-vendor path. Apache 2.0 code on a proprietary Kit SDK.

O3DE - OPEN 3D ENGINE

26.05.0, May 2026



Operational environments

Editor, prefabs and scripted behaviors model facilities and interacting subsystems.

Native ROS 2

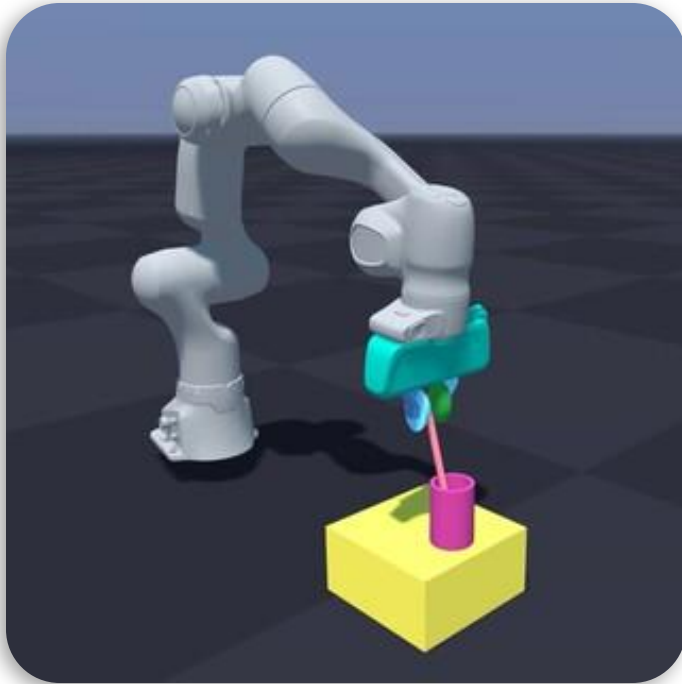
Custom messages, services and actions use ROS APIs directly. Built-in TF, time and namespaces, Nav2 and MoveIt.

Focused

No support for robot learning, great for dev / SiL / operational.

NEWTON

1.6.0, September 2026



OpenUSD + MuJoCo + Warp

Extensible physics

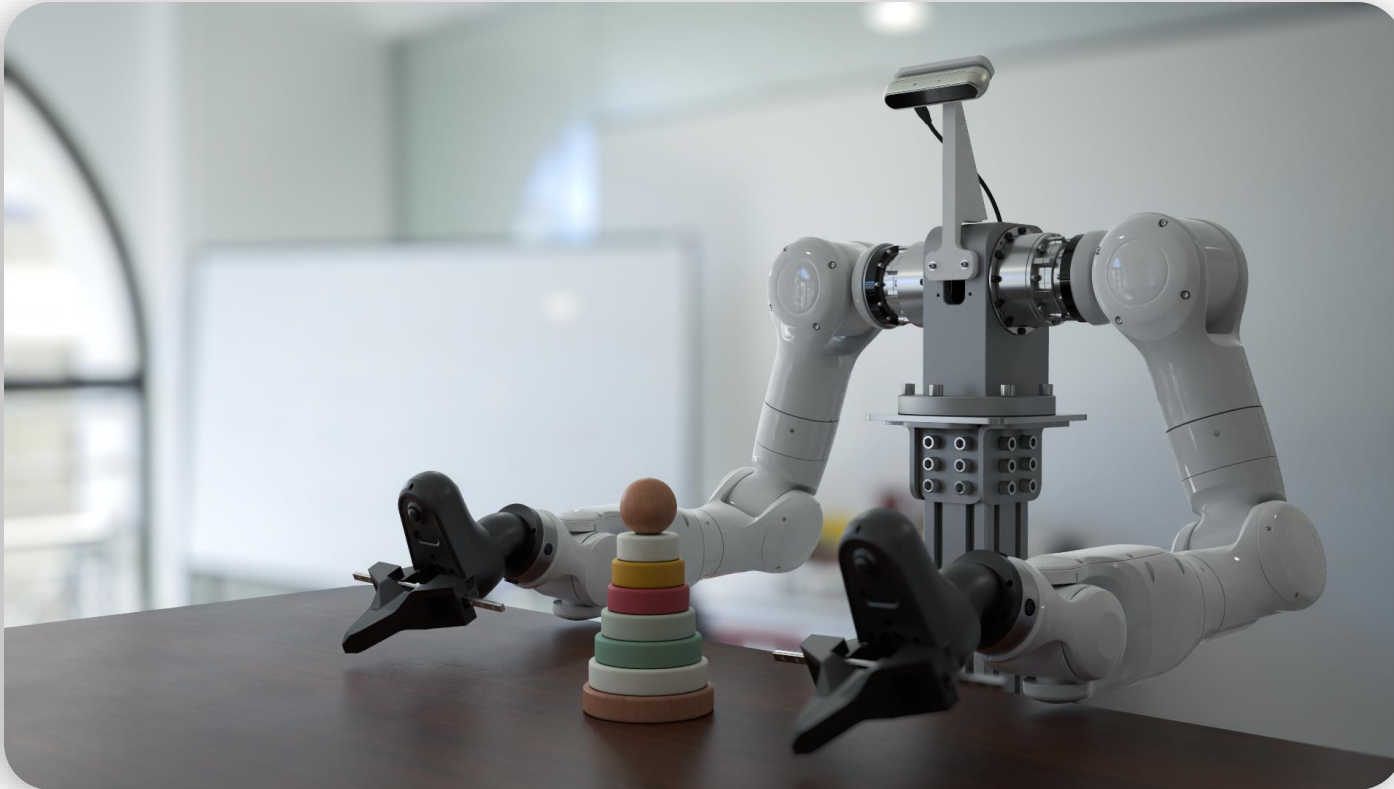
Warp kernels and solver selection, deformables, gradients, constraints, material coupling.

Under LF but backend lock-in

Requires Warp (CUDA jit) - NVIDIA backend.

GENESIS WORLD

1.4.1, September 2026



Coupled physics

Rigid, FEM, MPM and PBD/SPH solvers share one scene; explicit, SAP or IPC couplers resolve contact between them.

Task variation at scale

Parallel and heterogeneous environments, domain randomization, batched IK and planning.

Beyond state-only policies

Cameras (path-traced or raster), lidar, IMU, contact-force and tactile taxel grids; differentiable by design.

OTHER TOOLS: LEARNING AND EVALUATION

Schola

Unreal-to-Python RL integration with vectorized and multi-agent workflows. Useful for authored training environments.

SAPIEN + ManiSkill

Articulated-object manipulation, including visual observations. ManiSkill adds tasks, datasets and learning infrastructure.

LIBERO

Tests transfer across objects, spatial layouts and task goals. Useful when policy evaluation needs a defined manipulation suite.

OTHER TOOLS: ROBOTICS AND WORLD BUILDING

Drake

Manipulation with contact-rich physics, plus planning, trajectory optimization and controller design. Built for optimization rather than rollouts; gradients built in.

Webots

Robot models, sensors and controllers in an integrated environment. Useful for prototyping and repeatable controller experiments.

Unreal Engine

Detailed environments, behavior authoring and visual simulation. Matters for photorealism.

Unity

Existing projects can reuse ros2-for-unity (though no longer supported by Robotec.ai).

A SIMPLIFIED CHEAT SHEET

My problem	Candidates	Deciding factor
SiL stack tests	Gazebo, O3DE, Isaac Sim	Control, sensors, photorealism
Train a policy	MuJoCo / Playground, Genesis World, Isaac Lab	Task and compute backend
Large, rich scenes	O3DE, Unreal	Authoring and performance
Custom physics	Newton, Genesis World, MuJoCo (backends)	Required solver features
Policy benchmarks	LIBERO, ManiSkill task suites	Benchmark fit

STANDARDS, INTEROPERABILITY

Shared	Reusable	Enabling
Simulation workloads	Spawn, reset, step and test orchestration	Simulation Interfaces
Assets and scenes	Robot and environment descriptions	Interoperable OpenUSD REP 158
Runtime state	State exchanged between coupled simulations	Co-simulation: open gap (FMI, DCP are an underwhelming fit)

SIMULATION INTERFACES



Reusable test workflows

Interoperable scenario script controls spawning, entity state, reset and stepping.

Working examples

Gazebo, O3DE and Isaac Sim share the warehouse scene. RViz2 control plugin.

Broader adoption

Community Newton and Genesis World bridges implement subsets.

OpenUSD REP-158

In Vote

- | | | |
|---|--|---|
| 1 | Upstream standards and sim-ready assets | Adopt OpenUSD standard and SimReady practices, modular composition, neutral physics + optional simulator-specific layers. |
| 2 | Declarative ROS schemas | Topics, services, actions and frames in the asset, independent of the simulator's implementation. |
| 3 | Compatibility and downstream export | Enable efficient glTF and URDF export through material, geometry, instancing and variant rules. |
| 4 | A shared ecosystem | Maintain core schemas, extension registries and compliance tooling so assets can be checked and reused. |

BUILDING WITH AI AGENTS

Skills

Build skills for your simulator first – especially when versions carry large diffs (e.g. Gazebo / Ignition). Include skills to present you with renders / evidence.

Sim engineering

As with most code, but spatio-temporal reasoning has limits, and images cost input tokens. Have good validation methods and highest-signal artifacts to inspect.

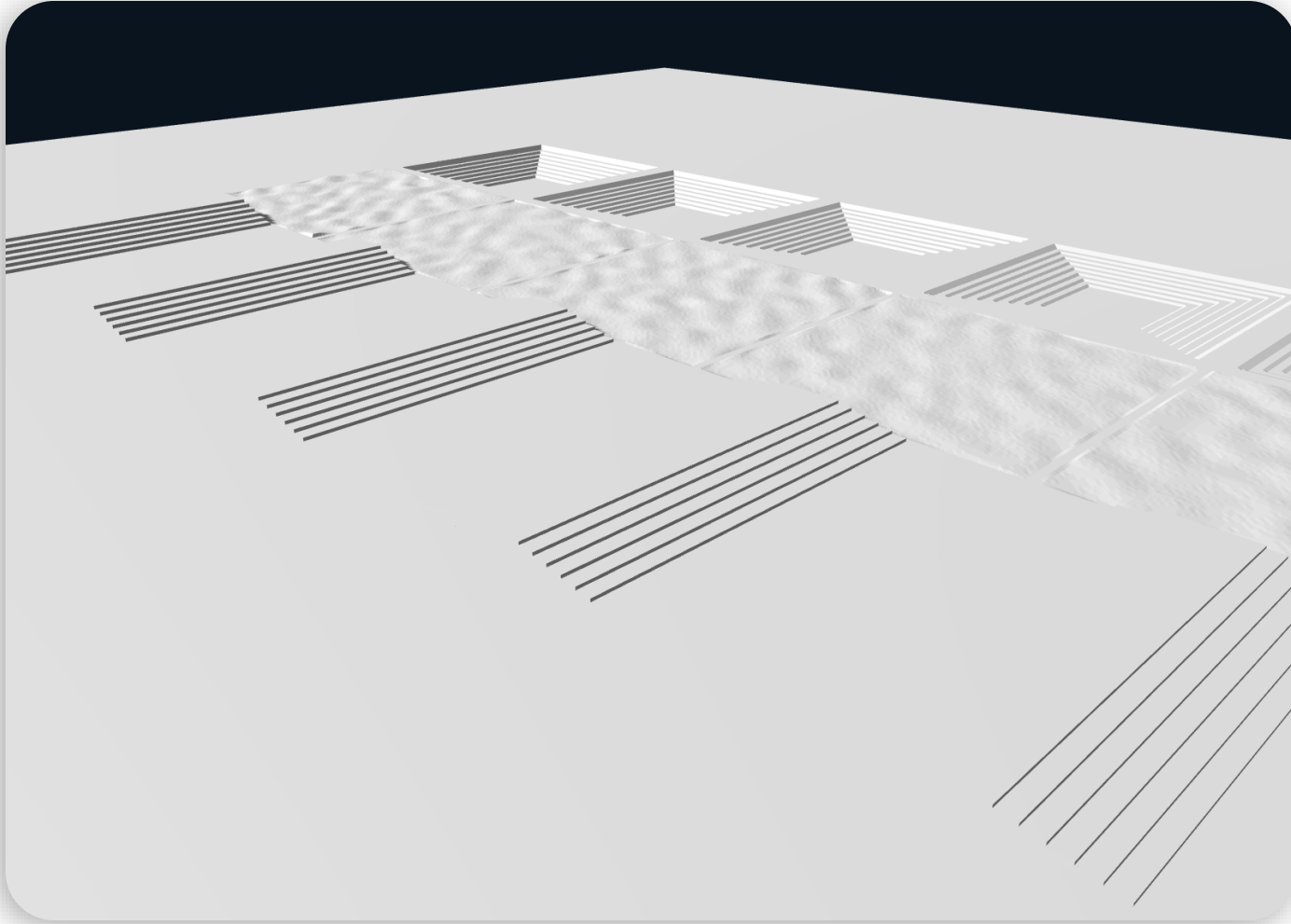
Scene assembly & scenario testing

Agents can quickly and quite successfully compose your assets into a reasonable scene. Use simulation interfaces.

3D model, scene generation

Simplified models can be generated from photos of the place. They are a good starting point. GPT-6 Astra outclasses other models here.

BACK TO OUR TRAINING

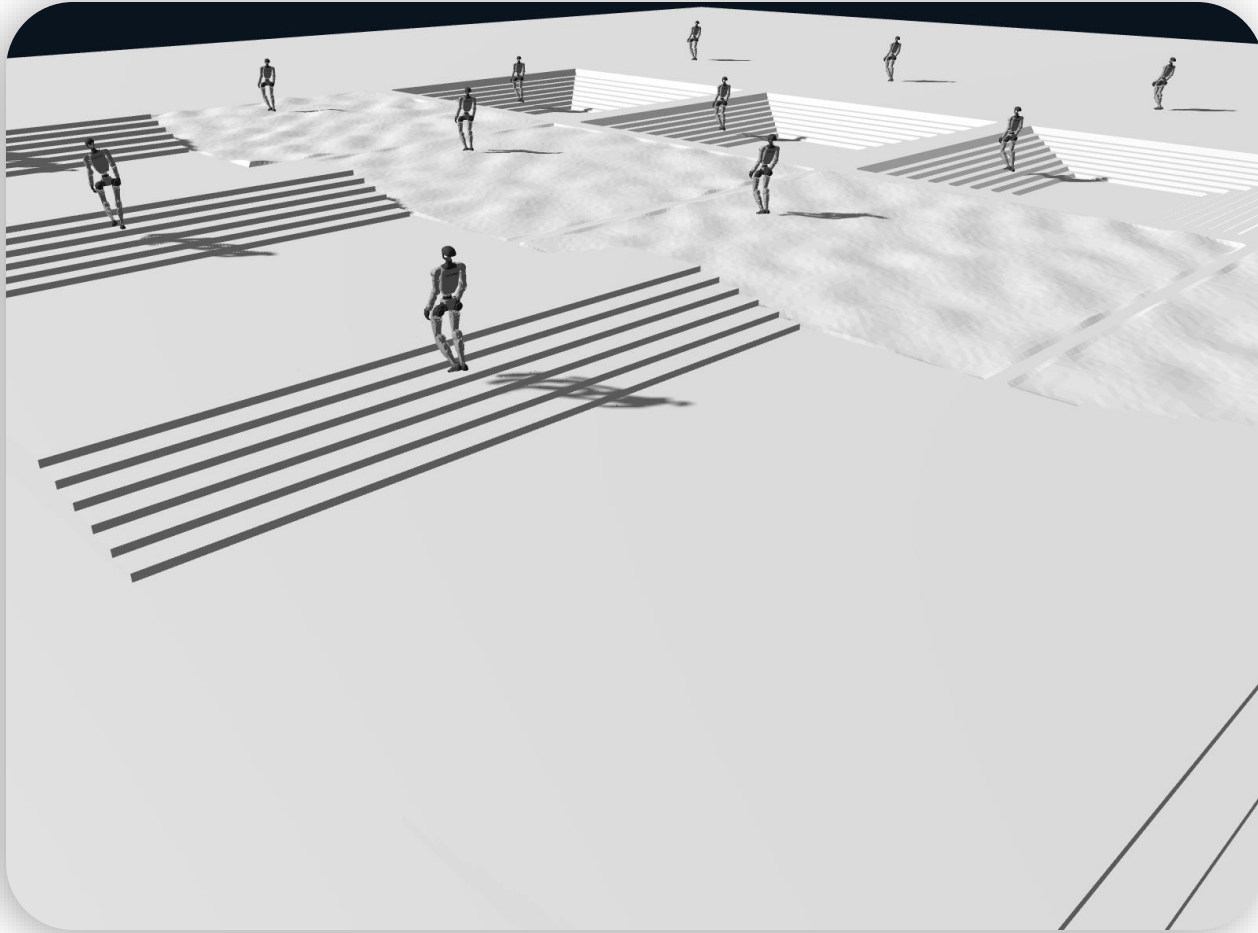


Started from a policy trained for flat ground.

Increasingly difficult stairs.

Compare both policies in the same evaluation setup.

CHANGES - TUNING



Terrain and curriculum

Introduce the new conditions and control their difficulty.

Rewards

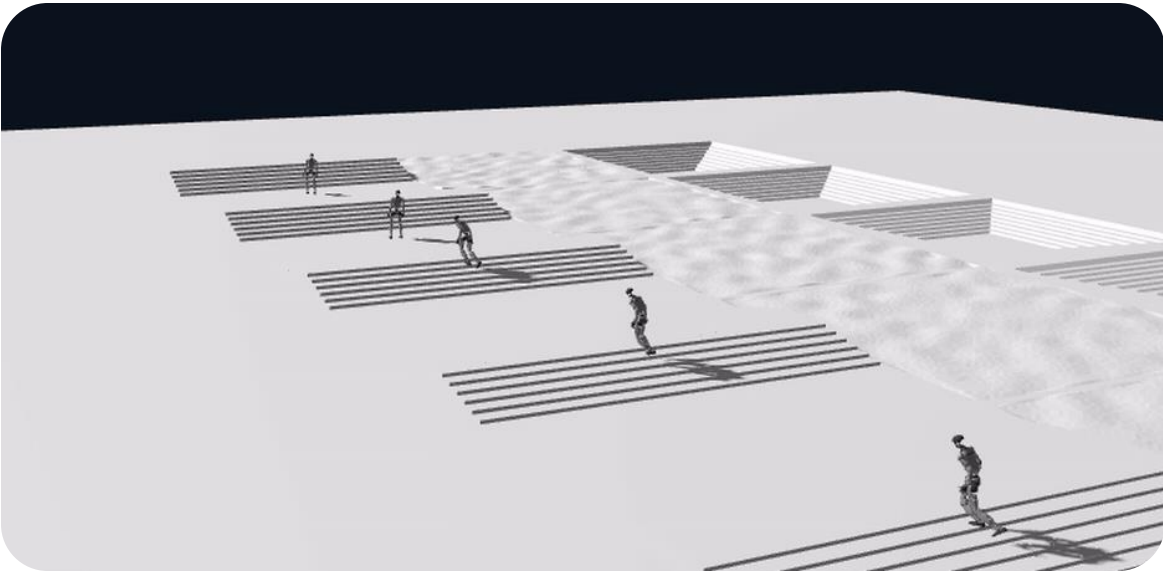
Revisit foot clearance and body orientation.

Observations

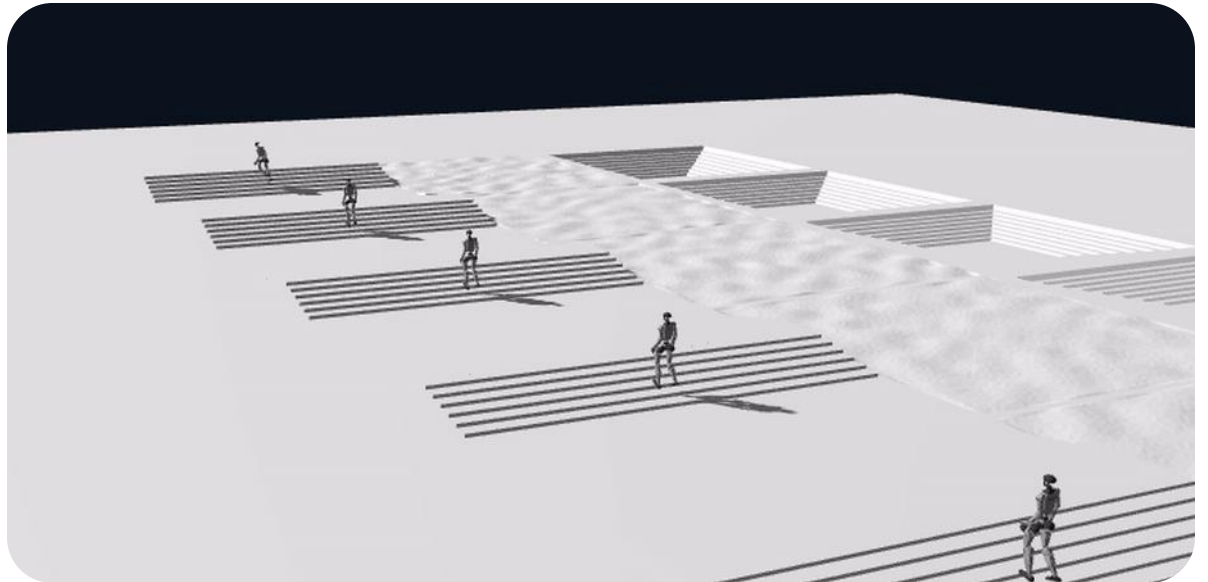
Review terrain and foot-height information available to the policy.

BASELINE AND FINE-TUNED POLICY

Baseline



Fine-tuned



Compare falls, foot clearance, slip and body orientation.

Measured outcome: fall rate across all terrain difficulty levels reduced from 24% to 1-2%

SHARING, Q&A

What is hard to simulate?

What did you use successfully?

What are the gaps for co-simulation?



Talk links and practice
resources